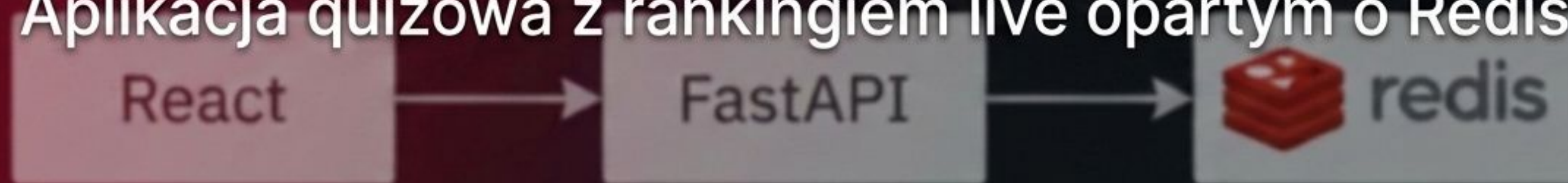


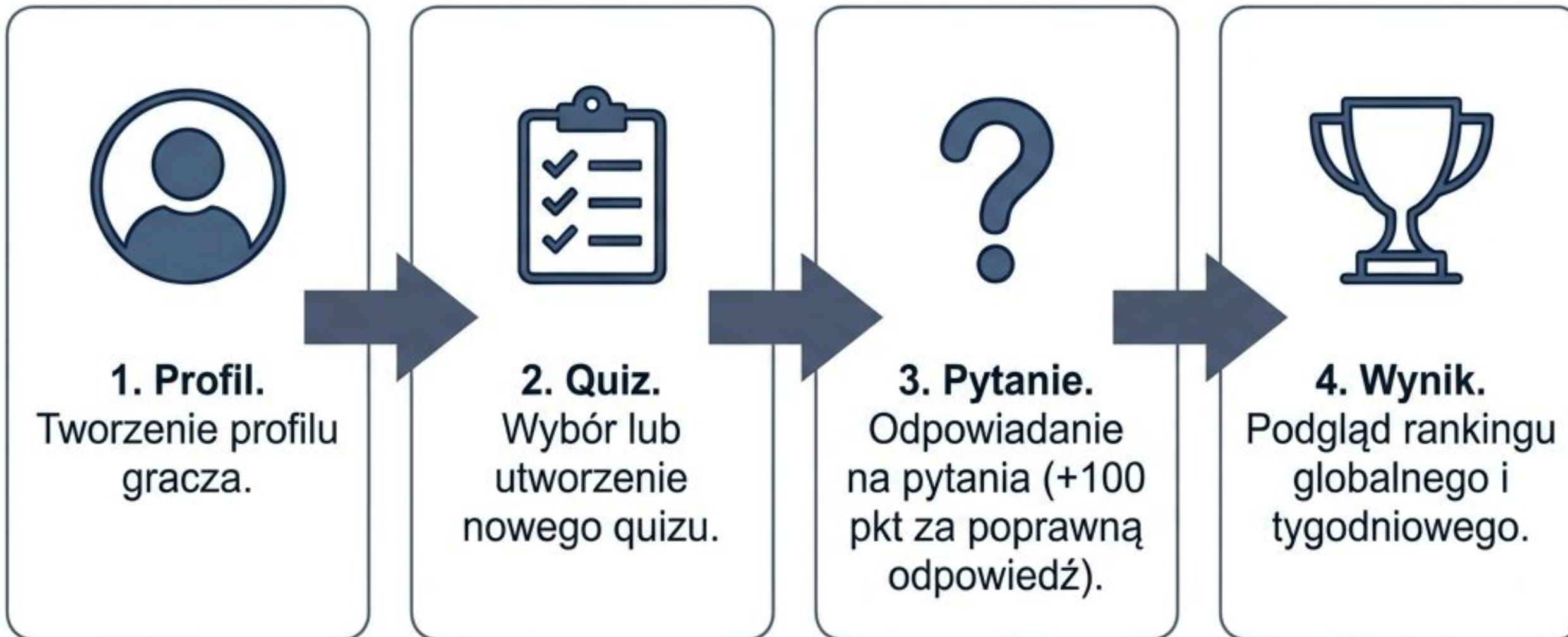
Quiz Leaderboard

Aplikacja quizowa z rankingiem live opartym o Redis



Hubert Marciniak
Mateusz Mańczak
Roch Zaremba

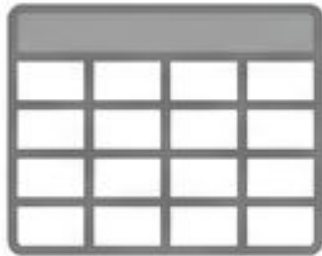
Ścieżka Użytkownika i Logika Gry



Kluczowa Cecha:
Aktualizacja
rankingów na
ekranach
wszystkich graczy
następuje bez
odświeżania strony.

Dlaczego Redis? (Relacje vs. In-Memory)

Bazy Relacyjne (SQL)



Sortowanie tysięcy wierszy zapytaniem ORDER BY przy każdej zmianie wyniku jest niezwykle kosztowne wydajnościowo.

Wymaga skomplikowanych operacji JOIN dla połączenia tabel użytkowników, wyników i quizów.

Redis (In-Memory NoSQL)

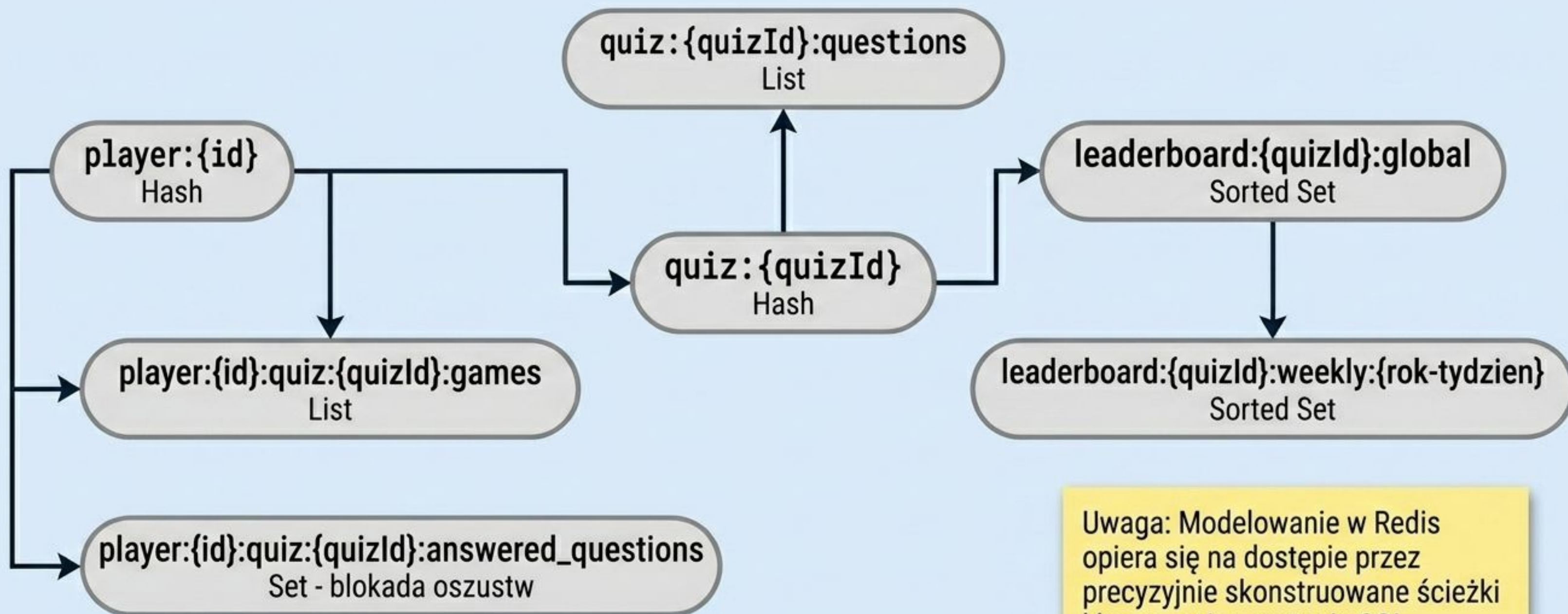


Operacje w pamięci RAM zapewniają błyskawiczną odpowiedź (Sub-ms read/write).

Wbudowane struktury danych naturalnie utrzymują posortowany ranking przy każdym dodaniu punktów.

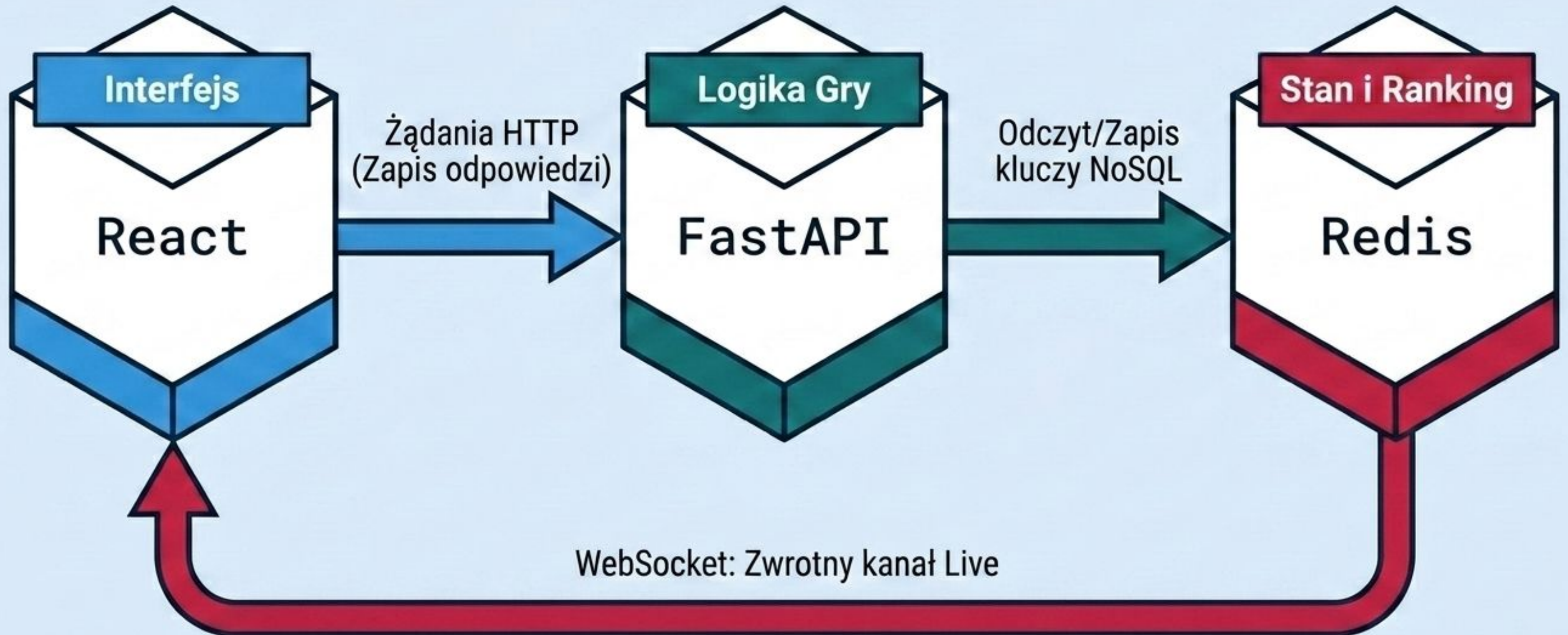
Złożoność algorytmiczna przeliczenia to zaledwie $O(\log(N))$.

Topografia Przestrzeni Kluczy

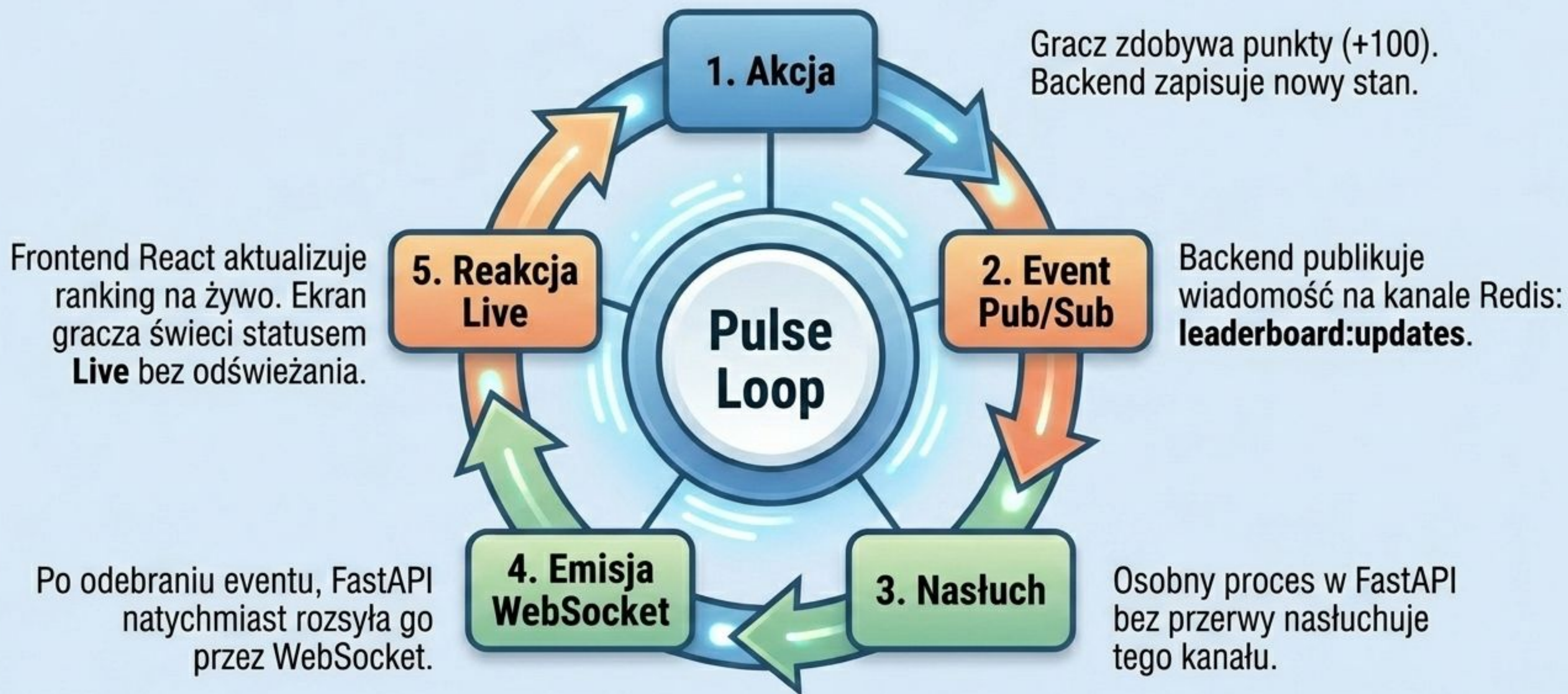


Uwaga: Modelowanie w Redis opiera się na dostępie przez precyzyjnie skonstruowane ścieżki kluczy, a nie zapytania SQL złączeniowe.

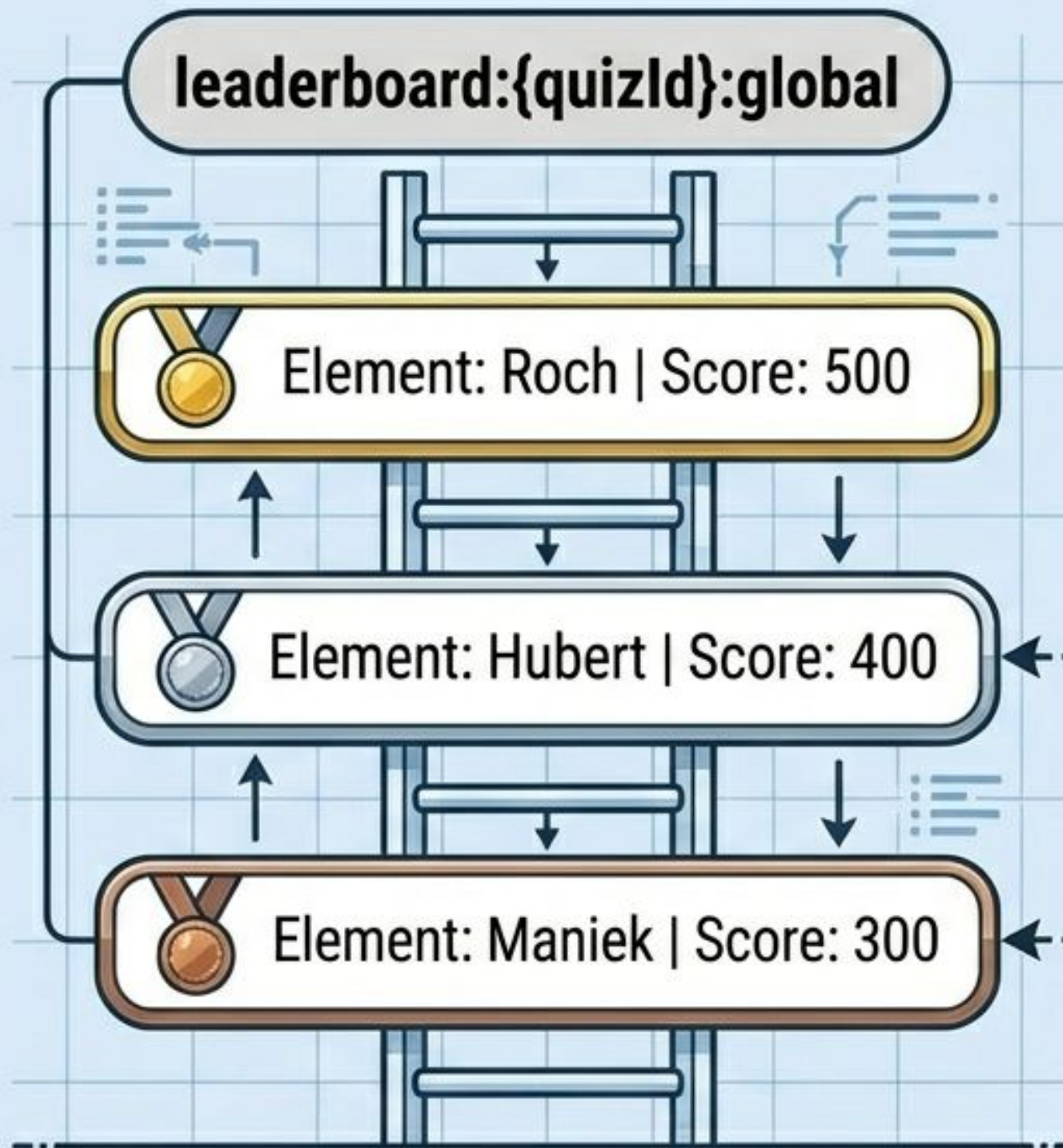
Architektura Warstwowa Systemu



Silnik Czasu Rzeczywistego (Pub/Sub)



Silnik Rankingu: Sorted Sets



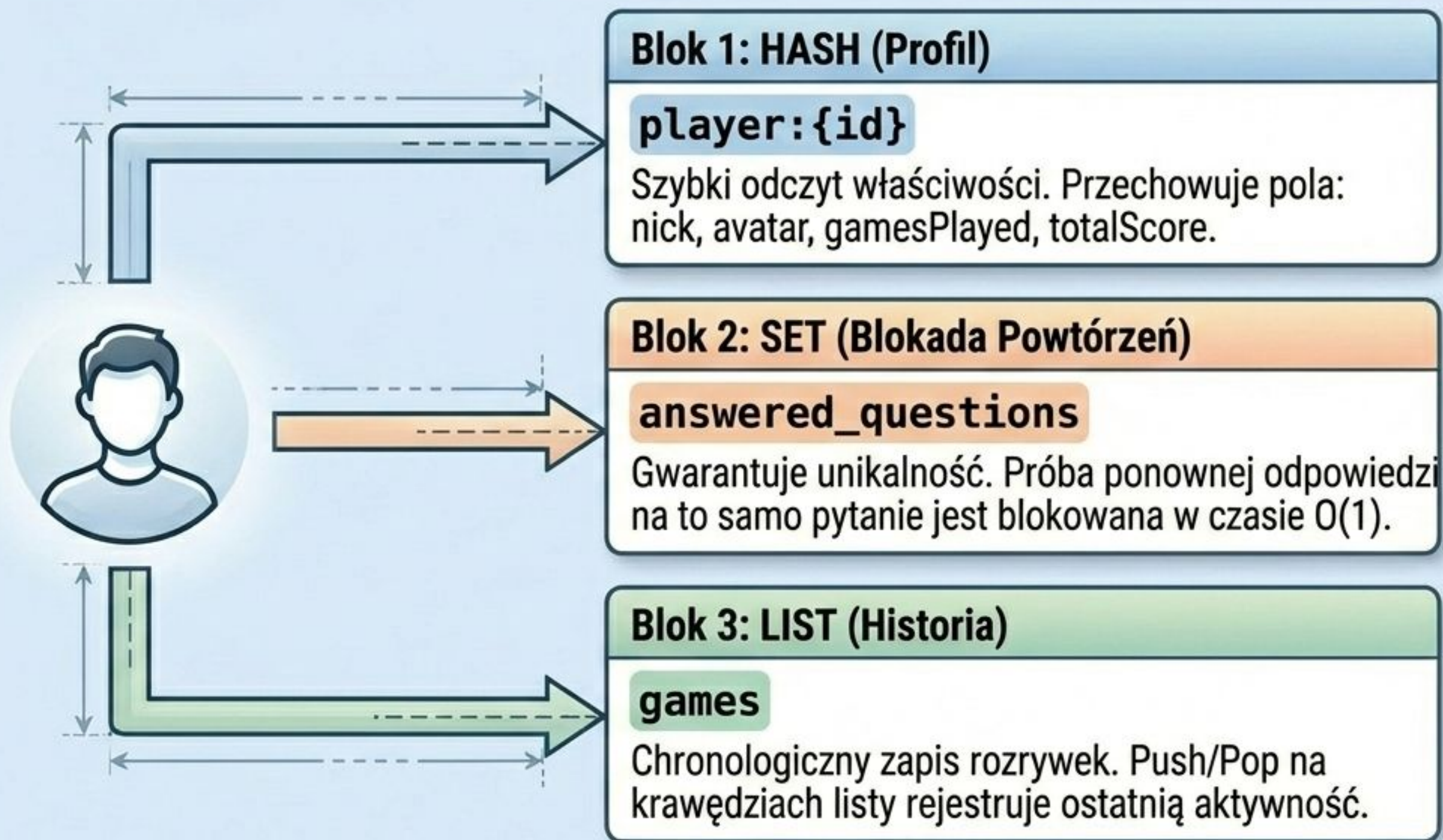
Mechanika ZINCRBY

Backend wykonuje komendę ZINCRBY. Redis automatycznie przelicza punkty i błyskawicznie przestawia elementy w czasie $O(\log(N))$.

Ranking Tygodniowy

Ten sam mechanizm działa dla klucza z dodanym czasem (np. `:weekly:2024-42`), co umożliwia generowanie szybkich statystyk okresowych bez dodatkowych obliczeń na backendzie.

Stan Gracza: Hash, Set i Lista



Matryca Architektury Danych

Wymaganie Biznesowe	Typ Redis	Przykład Klucza	Złożoność Obliczeniowa
Szybki ranking graczy	Sorted Set	leaderboard:global	$O(\log(N))$
Zabezpieczenie przed wielokrotnym punktowaniem	Set	answered_questions	$O(1)$
Profil gracza z wieloma atrybutami	Hash	player:123	$O(1)$
Historia ostatnich gier	List	games	$O(1)$ dla skrajnych elementów



Zapraszamy na demo